

Embracing Dynamics: Dynamics-aware 4D Gaussian Splatting SLAM

Zhicong Sun^{1,2}, Jacqueline Lo^{* 1} and Jinxing Hu^{*2}

Abstract—Simultaneous localization and mapping (SLAM) technology has recently achieved photorealistic mapping capabilities thanks to the real-time, high-fidelity rendering enabled by 3D Gaussian Splatting (3DGS). However, due to the static representation of scenes, current 3DGS-based SLAM encounters issues with pose drift and failure to reconstruct accurate maps in dynamic environments. To address this problem, we present D4DGS-SLAM, the first SLAM method based on 4DGS map representation for dynamic environments. By incorporating the temporal dimension into scene representation, D4DGS-SLAM enables high-quality reconstruction of dynamic scenes. Utilizing the dynamics-aware InfoModule, we can obtain the dynamics, visibility, and reliability of scene points, and filter out unstable dynamic points for tracking accordingly. When optimizing Gaussian points, we apply different isotropic regularization terms to Gaussians with varying dynamic characteristics. Experimental results on real-world dynamic scene datasets demonstrate that our method outperforms state-of-the-art approaches in both camera pose tracking and map quality.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) has been a cornerstone in robotics and computer vision, enabling applications such as autonomous navigation, augmented reality, and 3D reconstruction. Recently, SLAM has benefited from 3D Gaussian Splatting (3DGS) [7], which offers explicit map representation and real-time high-fidelity rendering. 3DGS-based methods such as SplaTAM [6], MonoGS [9], and GS-SLAM [24] have achieved state-of-the-art (SOTA) map reconstruction quality and speed in static environments. However, these methods face challenges in dynamic environments with moving objects, leading to camera pose estimation drift and the appearance of artifacts in the map, or even to map reconstruction failure, as shown in Fig. 1. Recent work such as DG-SLAM [24] improves tracking performance in dynamic environments by filtering dynamic objects. However, since it depends on a static 3DGS representation, it cannot fully reconstruct the entire scene nor produce high-quality complete maps.

Beyond 3DGS-based SLAM, SLAM methods based on Neural Radiance Fields (NeRF) [10] also provide photorealistic maps. Following the demonstration of high-quality map reconstruction capabilities by NeRF-based SLAM methods such as iMAP [16], NICE-SLAM [30], ESLAM [5] and Co-SLAM[20], approaches like DN-SLAM [14], NID-SLAM [23], DDN-SLAM [8], and RodYN-SLAM [4] have



Fig. 1. Comparison of rendering results between 3DGS-based MonoGS and our 4DGS-based D4DGS-SLAM on Bonn and TartanAir-Shibuya Datasets.

extended NeRF’s capabilities to dynamic scenes. Although these methods achieve map reconstruction quality comparable to 3DGS-based methods, their real-time performance is hindered by the significant computational demands of ray tracing used in map rendering.

To achieve accurate camera pose estimation and real-time high-fidelity map reconstruction in dynamic scenes, we propose DD4DGS-SLAM. To the best of our knowledge, this work presents the first SLAM system based on a dynamic map representation. Unlike existing 3DGS-based SLAM methods that filter out dynamic objects, we choose to “embrace” dynamics by using 4D Gaussian Splatting (4DGS) [26] with a temporal dimension for map representation and rendering. 4DGS enables our system to effectively learn the spatio-temporal characteristics of the scene, thereby reconstructing a map with few artifacts without filtering out dynamic objects. To reduce tracking drifts in dynamic environments when using photometric and geometric error-based optimization, we introduce a long-term effective any point tracking (LEAP) [3] for dynamics-aware tracking point filtering. By leveraging the dynamics and reliability of anchors in the scene provided by LEAP, we can select stable and static points in the current frame for tracking, significantly improving the accuracy of camera pose estimation. Dynamic awareness also facilitates the densification and pruning of Gaussians over time, allowing the Gaussian

* Corresponding authors: Jinxing Hu and Jacqueline Lo.

¹ Hong Kong Polytechnic University, Hong Kong SAR, China.

² Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China.

⁺ Emails: zhicong.sun@connect.polyu.hk, jinxing.hu@siat.ac.cn, jacqueline.lo@polyu.edu.hk

set to better fit dynamic scenes. The experimental results on real-world dynamic datasets demonstrate that our method achieves SOTA performance in both camera pose estimation and mapping quality. Our contributions are summarized as follows:

- 1) We propose the first SLAM system based on a 4DGS with dynamic scene representation capabilities, enabling spatio-temporal modeling of dynamic environments.
- 2) We enhance the SLAM system with the dynamics-aware InfoModule, upon which we build strategies and a pipeline for tightly integrating dynamic information, significantly improving tracking and mapping capabilities.
- 3) Our method achieves SOTA tracking and mapping performance on multiple dynamic SLAM benchmarks and demonstrates the capability to reconstruct complete maps of dynamic scenes under precise pose estimation.

II. RELATED WORK

Early SLAM systems laid the foundation for real-time localization and mapping through geometric feature tracking. Seminal works like ORB-SLAM2 [11] and ORB-SLAM3 [2] established feature-based pipelines using sparse hand-crafted descriptors. These methods excel in static environments but suffer severe degradation when dynamic objects dominate observations, as moving features introduce false geometric constraints. Subsequent improvements like DS-SLAM [27], DynaSLAM [1], VDO-SLAM [28], FlowFusion [29], Raft [18], Gmflow [21] attempted to address this through semantic segmentation masks or optical flow, yet remained limited either by their dependency on pre-trained recognition models or inability to handle unknown moving entities. The fundamental constraint of these systems lies in their static world assumption.

The emergence of Neural Radiance Fields (NeRF) [10] catalyzed a paradigm shift towards continuous scene representations. Pioneering neural SLAM systems like iMAP [16], NICE-SLAM [30], ESLAM [5] and Co-SLAM[20] combined implicit geometry encoding with simultaneous pose optimization, achieving unprecedented reconstruction quality. However, their volumetric ray-marching pipelines incur heavy computational costs, making them impractical for real-time applications. More critically, these methods inherently entangle static and dynamic elements within their MLP weights, causing catastrophic mapping failures when objects move during camera observation. While recent works like DN-SLAM [14], NID-SLAM [23], DDN-SLAM [8], and RodYN-SLAM [4] attempt dynamic and static decomposition through optical flow or semantic segmentation, their neural rendering backbone remains too slow for interactive SLAM. This performance-realism tradeoff motivated the community to seek alternative representations.

3DGS [7] revolutionized scene representation through explicit, differentiable rasterization. By modeling scenes as anisotropic Gaussians with view-dependent appearance, 3DGS achieves photorealistic rendering at 200+ FPS while

maintaining explicit geometric control. This breakthrough inspired Gaussian-based SLAM systems like SplatAM [6], MonoGS [9] and GS-SLAM [24], which optimize Gaussian parameters alongside camera poses. Though these methods set new SOTA in static scene reconstruction, they inherit the static assumption bottleneck, namely dynamic objects cause irreversible corruption of the Gaussian map due to unmodeled temporal variations. Despite efforts by recent works such as DG-SLAM [24] to tackle tracking issues in dynamic environments by filtering out dynamic objects, the reliance on a static 3DGS representation limits its ability to fully reconstruct an accurate map. Unlike the aforementioned works, our approach embraces dynamic objects and learns their characteristics rather than removing them. As demonstrated below, this concept significantly enhances the capabilities of the SLAM system in dynamic environments.

III. METHOD

The pipeline of D4DGS-SLAM is illustrated in Fig. 2. Our system uses an RGB-D image sequence as input. We first extract anchors from each incoming RGB frame that are well-distributed globally and reflect the image features. These anchors, along with the RGB images, are fed into the LEAP module to obtain the dynamics and reliability of the anchors. This allows us to distinguish between stable dynamic points and static points. The static anchors are used for tracking to estimate the camera pose. These poses and dynamic information are then sent to the mapping module. We use 4DGS for mapping and select different scale penalty factors based on the dynamics and reliability of the covered points to control the distribution of the Gaussian in space-time. The techniques used and our SLAM system will be introduced below.

A. 4D Gaussian Map Representation

Our SLAM system represents a dynamic scene by a set of anisotropic 4D Gaussians. Each Gaussian captures the spatio-temporal characteristics of a region in the scene. The unnormalised probability density function of a 4D Gaussian is given by:

$$p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}, \quad (1)$$

where $\mathbf{x} = (x, y, z, t)$ denotes a point in the 4D spatio-temporal space, $\boldsymbol{\mu} = (\mu_x, \mu_y, \mu_z, \mu_t)$ is the mean vector representing the Gaussian's position in space and time, and $\boldsymbol{\Sigma} \in \mathbb{R}^{4 \times 4}$ is the covariance matrix describing its shape and orientation and encoding the Gaussian's spatial extent and temporal duration. $\boldsymbol{\Sigma}$ is parameterized as $\boldsymbol{\Sigma} = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T$, where $\mathbf{S} = \text{diag}(s_x, s_y, s_z, s_t)$ is the scaling matrix, and $\mathbf{R} = L(q_l)R(q_r)$ is the 4D rotation matrix constructed from two quaternions q_l and q_r . The 4D Gaussian can be decomposed into a conditional 3D Gaussian $p(x, y, z|t) = \mathcal{N}((x, y, z); \boldsymbol{\mu}_{xyz|t}, \boldsymbol{\Sigma}_{xyz|t})$ for spatial distribution and a marginal 1D Gaussian $p(t) = \mathcal{N}(t; \mu_t, \Sigma_{t,4})$ for temporal distribution. The conditional mean $\boldsymbol{\mu}_{xyz|t}$ and covariance

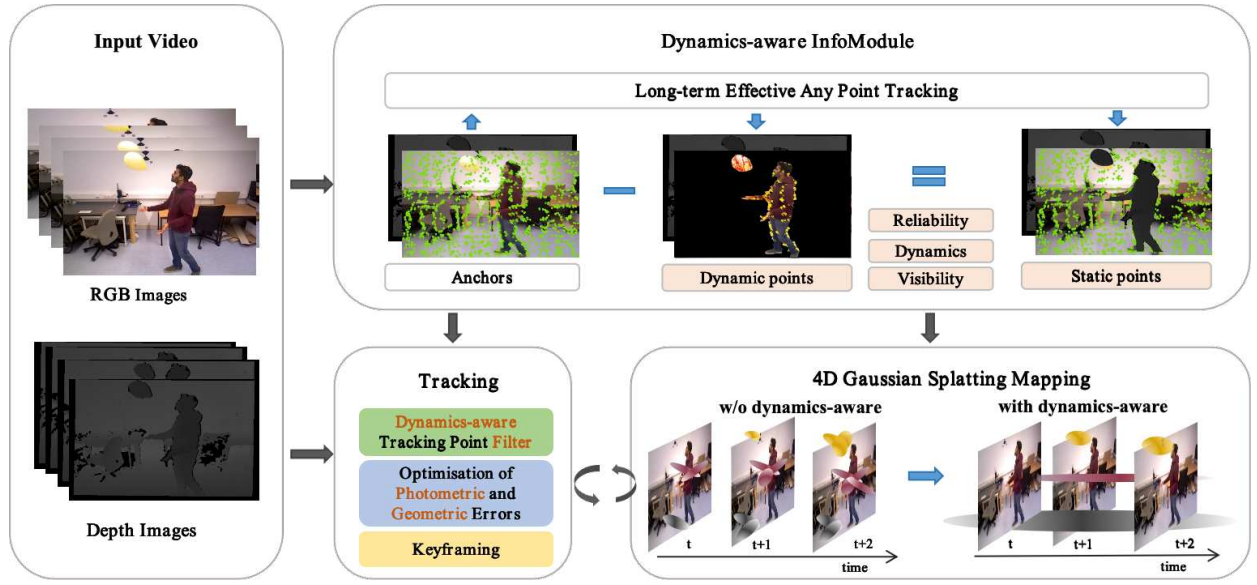


Fig. 2. Overview of D4DGS-SLAM

$\Sigma_{xyz|t}$ are derived as:

$$\begin{aligned}\mu_{xyz|t} &= \mu_{1:3} + \Sigma_{1:3,4} \Sigma_{4,4}^{-1} (t - \mu_t), \\ \Sigma_{xyz|t} &= \Sigma_{1:3,1:3} - \Sigma_{1:3,4} \Sigma_{4,4}^{-1} \Sigma_{4,1:3}.\end{aligned}\quad (2)$$

In the map rendering process, the color of a pixel (u, v) at time t is computed by blending the contributions of visible 4D Gaussians. This rendering process ensures smooth transitions in both space and time. The rendering equation is formulated as:

$$\begin{aligned}\mathcal{I}(u, v, t) &= \sum_{i=1}^N p_i(t) p_i(u, v|t) \alpha_i c_i(d, t) \\ &\times \prod_{j=1}^{i-1} (1 - p_j(t) p_j(u, v|t) \alpha_j),\end{aligned}\quad (3)$$

Here, $p_i(t)$ is the marginal temporal distribution of the i -th Gaussian, derived from the 4D Gaussian's temporal component as $p_i(t) = \mathcal{N}(t; \mu_{t,i}, \Sigma_{t,i})$. The term $p_i(u, v|t)$ is the conditional spatial distribution of the i -th Gaussian, obtained by projecting the 3D Gaussian $p_i(x, y, z|t)$ onto the image plane, where $p_i(u, v|t) = \mathcal{N}((u, v); \mu_{uv|t,i}, \Sigma_{uv|t,i})$. The opacity α_i is directly obtained from the Gaussian's optical properties, while the view-dependent color $c_i(d, t)$ is represented using 4D Spherindrical Harmonics (4DSH) as that in [26]. This decomposition enables the 4D Gaussian to separately model the spatial and temporal characteristics of dynamic scenes, which is crucial for the rendering process.

B. Long-term Effective Any Point Tracking

Our SLAM system integrates the pretrained LEAP module [3] to robustly track query points across image sequences. LEAP provides three highly beneficial pieces of information for dynamic SLAM: visibility, dynamic status, and reliability of visibility for each tracked point in the current frame.

Considering a sequence of S consecutive RGB images $\mathbf{I} = [\mathbf{I}_1, \dots, \mathbf{I}_S]$, LEAP predicts the trajectory $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_S]$, dynamic track label $\mathbf{M}_d = [\mathbf{m}_{d1}, \dots, \mathbf{m}_{dS}]$, visibility $\mathbf{V} = [v_1, \dots, v_S]$ and reliability of visibility $\Phi = [\phi_1, \dots, \phi_S]$ for a query point $\mathbf{x}_q$ in frame s_q . This can be expressed as:

$$(\mathbf{X}, \mathbf{M}_d, \mathbf{V}, \Phi) = \text{LEAP}(\mathbf{I}, \mathbf{x}_q, s_q). \quad (4)$$

Visibility indicates whether a tracking point is visible or occluded in a given frame. LEAP predicts the visibility of each point throughout the sequence as a binary label $v_s \in \{0, 1\}$, where $v_s = 1$ indicates the point is visible in frame s , and $v_s = 0$ indicates occlusion. The visibility is derived from the final point feature $\mathbf{F}^K$ after K iterations of refinement, using a simple linear projection layer $\mathcal{G}_v$:

$$\mathbf{V} = \mathcal{G}_v(\mathbf{F}^K), \quad (5)$$

Dynamic status determines whether a point belongs to a static or dynamic object in the scene. LEAP predicts the dynamic label $\mathbf{m}_d$ for each trajectory by leveraging both visual appearance and long-term motion cues. Specifically, LEAP tracks additional anchors alongside the original queries to capture global motion patterns. The anchors are selected by the gradient-based sampling method. Fig. 3 visualizes the anchor points, which are widely distributed in regions with high gradient magnitudes. These anchor points are utilized during tracking initialization and keypoint addition, ensuring robust and distinctive point selection for subsequent tracking. The dynamic label is estimated using a shallow MLP layer $\mathcal{G}_d$ with average pooling over the temporal dimension:

$$\mathbf{m}_d = \text{avgpool}(\mathcal{G}_d([\mathbf{X}^K; \mathbf{X}_A^K], [\mathbf{F}^K; \mathbf{F}_A^K])), \quad (6)$$

where $\mathbf{X}^K$ and $\mathbf{F}^K$ are the final point trajectory and features, respectively, and $\mathbf{X}_A^K$ and $\mathbf{F}_A^K$ are the corresponding anchor trajectory and features.

Reliability of visibility quantifies the confidence in the visibility prediction. LEAP incorporates a probabilistic formulation to model the uncertainty of point trajectories. The uncertainty $\phi(\mathbf{x}_s)$ for each point is derived from the scale matrices Σ_a and Σ_b of the multivariate Cauchy distribution:

$$\phi(\mathbf{x}_s) = \Sigma_a[s, s] + \Sigma_b[s, s], \quad (7)$$

where Σ_a and Σ_b are the scale matrices for the X and Y coordinates, respectively. Points with low uncertainty (high reliability) are prioritized for tracking and optimization.



Fig. 3. Visualization of anchors.

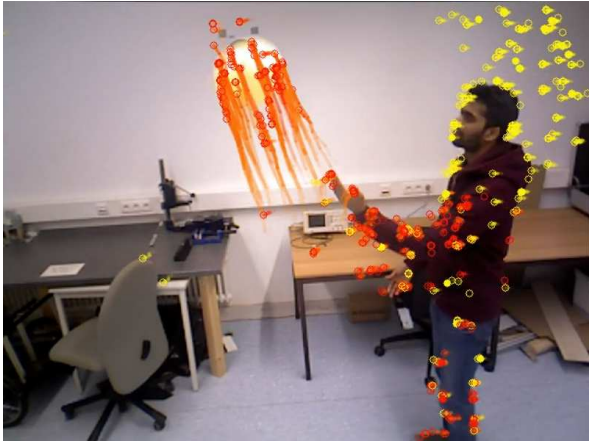


Fig. 4. Anchors with high dynamics (in red) and low reliability (in yellow)

Fig. 4 illustrates the highly dynamic points and points with uncertain dynamic status in the current frame, as identified by the LEAP module. This visualization highlights the ability of LEAP to distinguish between dynamic and uncertain regions, which is crucial for accurate tracking and mapping in dynamic environments. Unlike other pairwise trackers, LEAP is capable of long-term point tracking across image sequences, demonstrating SOTA performance in dynamic environments. This capability ensures its effectiveness in camera pose estimation and dynamic environment mapping within SLAM systems. Our experimental results further validate this claim.

C. SLAM System

1) *Dynamics-aware Tracking*: In the tracking stage of our SLAM system, we focus on updating the list of tracked points and estimating the camera pose without performing map updates. During initialization, we first use Sobel kernels to obtain the RGB image gradients and apply an average pooling layer to the gradient map to smooth and downsample it. Next, we divide the gradient map into multiple subregions and select the point with the highest image gradient magnitude in each subregion as anchors. The anchors in first frame are selected as tracking anchors. Then the tracking process begins by leveraging the pre-trained LEAP model to obtain the 2D positions, visibility, dynamic status, and reliability of the tracked points in the current frame. Based on predefined thresholds, we filter out points with low visibility, high dynamic status, low reliability, or insufficient temporal observations, resulting in a stable list of static and trackable points. Specifically, we retain points with visibility $V_t \geq \gamma_v$, where γ_v is the visibility threshold, and discard points with dynamic status $\mathbf{m}_{d,t} \geq \gamma_d$, where γ_d is the dynamic status threshold. Additionally, we keep points with reliability $\Phi_t \geq Q(\gamma_u)$, where γ_u is the reliability quantile and Q is the quantile function, and remove points with fewer than γ_{track} valid observations across the sequence. If the number of tracked points falls below a predefined threshold, we extract new keypoints from $\mathbf{I}_t$ using the distributed image gradient-based sampling method and add them to the tracking list.

Then, camera pose estimation is performed on the basis of filtered tracked points, ensuring robustness and accuracy in dynamic environments. We minimize a combination of photometric and geometric residuals, computed exclusively for the tracked points that pass the filtering stage. This approach leverages both color and depth information from the RGB-D camera, while avoiding the influence of unreliable or dynamic points.

The photometric residual measures the difference between the rendered image and the observed image. For the filtered stable and static tracked points, the photometric residual is defined as:

$$E_{pho} = \|I_f(\mathcal{G}, \mathbf{T}_{CW}) - \bar{I}_f\|_1, \quad (8)$$

where $I_f(\mathcal{G}, \mathbf{T}_{CW})$ renders the Gaussians $\mathcal{G}$ from the current camera pose $\mathbf{T}_{CW}$, and $\bar{I}_f$ is the observed image.

The geometric residual measures the difference between the rendered depth and the observed depth. For the filtered stable and static tracked points, the geometric residual is defined as:

$$E_{geo} = \|D_f(\mathcal{G}, \mathbf{T}_{CW}) - \bar{D}_f\|_1, \quad (9)$$

where $D_f(\mathcal{G}, \mathbf{T}_{CW})$ is the depth rasterization from the current camera pose $\mathbf{T}_{CW}$, and $\bar{D}_f$ is the observed depth.

Finally, the camera pose is optimized by minimizing a weighted combination of the photometric and geometric residuals:

$$E_{total} = \lambda_{pho} E_{pho} + (1 - \lambda_{pho}) E_{geo}, \quad (10)$$

where λ_{pho} is a hyperparameter balancing the contributions of the photometric and geometric residuals.

By computing residuals exclusively for the filtered tracked points, our camera pose estimation method achieves robustness in dynamic environments while leveraging the full capabilities of the RGB-D camera. This approach ensures accurate and reliable pose estimation, even in the presence of challenging conditions such as occlusions or dynamic objects.

2) *Keyframing*: Our system adopts a keyframing strategy similar to MonoGS [9], which involves keyframe selection based on covisibility and relative translation, keyframe management to maintain a small window of non-redundant keyframes, and Gaussian insertion and pruning to refine the scene representation. A key distinction in our approach is the use of 4D Gaussians, which incorporate temporal information. When computing covisibility, we set the time parameter to the current frame's timestamp, enabling us to obtain the conditional 3D Gaussians distributed in space at the given time.

3) *Dynamics-aware 4DGS Mapping*: The purpose of mapping is to maintain a coherent spatial-temporal structure and to optimise the newly inserted Gaussians. During mapping, the keyframes in $\mathcal{W}_k$ are used to reconstruct currently visible regions. Additionally, two random past keyframes $\mathcal{W}_r$ are selected per iteration to avoid forgetting the global map.

In our method, we leverage dynamic status obtained from the front-end to distinguish between Gaussians with strong dynamics and those with weak dynamics. For Gaussians with strong dynamics, we apply an isotropic regularisation term that considers both spatial and temporal scaling parameters. For Gaussians with weak dynamics, the isotropic regularisation term only considers spatial scaling parameters. This approach allows us to use fewer Gaussians to represent relatively static parts of the scene while adding more Gaussians in dynamic regions to capture their spatiotemporal features more effectively. The isotropic regularisation term for Gaussians with strong dynamics is defined as:

$$E_{iso}^{dynamic} = \sum_{i=1}^{|\mathcal{G}^{dynamic}|} \left\| \vec{S}_i^{4D} - \tilde{\vec{S}}_i^{4D} \cdot \mathbf{1} \right\|_1 \quad (11)$$

where $\vec{S}_i^{4D}$ represents the spatiotemporal scaling parameters of a 4D gaussian, and $\tilde{\vec{S}}_i^{4D}$ is the corresponding mean. For Gaussians with weak dynamics, the isotropic regularisation term is defined as:

$$E_{iso}^{static} = \sum_{i=1}^{|\mathcal{G}^{static}|} \left\| \vec{S}_i^{3D} - \tilde{\vec{S}}_i^{3D} \cdot \mathbf{1} \right\|_1 \quad (12)$$

where only the spatial scaling parameters $\vec{S}_i^{3D}$ are considered.

Let the union of the keyframes in the current window and the randomly selected ones be $\mathcal{W} = \mathcal{K} \cup \mathcal{R}$. For mapping,

we solve the following optimisation problem:

$$\min_{T_{CW}^k \in SE(3), \mathcal{G}, \forall k \in \mathcal{W}} E_{total}^k + \lambda_{iso}^{dynamic} E_{iso}^{dynamic} + \lambda_{iso}^{static} E_{iso}^{static} \quad (13)$$

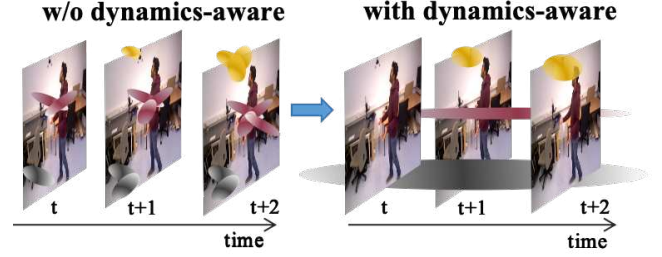


Fig. 5. Comparison of Gaussian distributions with and without dynamics-aware InfoModule

Figure 5 compares Gaussian Distributions with and without the dynamics-aware InfoModule. If we apply the same 4D isotropic regularisation to all Gaussians, multiple 4D Gaussians may exist even in the static parts of the map. However, when we use dynamics-aware isotropic regularisation, the distinction between dynamic and static regions results in a more reasonable distribution of Gaussians along the time axis. In the static parts, the Gaussians have a wide temporal width, while in the dynamic parts, the temporal distribution of Gaussians fits the occurrence of objects well.

TABLE I
RESULTS OF METRIC ATE RMSE ON SEVERAL DYNAMIC SCENE SEQUENCES IN BONN DATASET. “*” DENOTES THE VERSION REPRODUCED BY NICE-SLAM. “-” DENOTES THE TRACKING FAILURES. THE METRIC UNIT IS [CM].

Method	ball	ball2	ps.tk	ps.tk2	ball.tk	Avg.
ORB-SLAM3 [2]	5.8	17.7	70.7	77.9	3.1	29.8
ReFusion [12]	17.5	25.4	28.9	46.3	30.2	27.7
DROID-VO [19]	5.4	4.6	21.4	46.0	8.9	15.4
iMAP*[17]	14.9	67.0	28.3	52.8	24.8	36.1
NICE-SLAM[30]	-	66.8	54.9	45.3	21.2	44.1
Vox-Fusion[25]	65.7	82.1	128.6	162.2	43.9	88.4
Co-SLAM[20]	28.8	20.6	61.0	59.1	38.3	46.3
ESLAM[5]	22.6	36.2	48.0	51.4	12.4	31.4
Rodyn-SLAM[4]	7.9	11.5	14.5	13.8	13.3	12.3
SpliTAM[6]	35.5	36.1	149.7	91.2	12.5	57.4
GS-SLAM[24]	37.5	26.8	46.8	50.4	31.9	33.1
DG-SLAM [22]	3.7	4.1	4.5	6.9	10.0	5.5
Ours	3.6	3.9	4.5	5.2	8.5	5.1

IV. EXPERIMENT

A. Experimental Setup.

To evaluate the tracking and mapping performance of our method, we selected the challenging real-world dynamic SLAM datasets BONN datasets [12] and the more complex TartanAir-Shibuya dataset [13]. We adopt widely used metrics for camera tracking accuracy and map quality as in [15]. For camera tracking, we report the Root Mean Square Error (RMSE) of the Absolute Trajectory Error (ATE) in centimeters. For map quality, we report photometric rendering metrics, including Peak Signal-to-Noise Ratio (PSNR),

TABLE II
CAMERA TRACKING AND MAPPING QUALITY ON SEVERAL DYNAMIC SEQUENCES IN THE *TartanAir-Shibuya* DATASET.

	sh01				rc03				rc06				rc07				Avg.			
	ATE↓	PSNR↑	SSIM↑	LPIPS↓	ATE↓	PSNR↑	SSIM↑	LPIPS↓	ATE↓	PSNR↑	SSIM↑	LPIPS↓	ATE↓	PSNR↑	SSIM↑	LPIPS↓	ATE↓	PSNR↑	SSIM↑	LPIPS↓
SplaTAM [6]	64.0	12.02	0.310	0.457	52.6	11.11	0.234	0.621	82.8	13.33	0.513	0.358	93.4	13.25	0.477	0.509	73.2	12.43	0.384	0.486
MonoGS [9]	53.0	13.91	0.303	0.576	58.8	13.21	0.241	0.638	82.8	16.63	0.516	0.404	85.6	14.74	0.489	0.534	70.1	14.62	0.387	0.538
Ours	3.2	23.48	0.673	0.268	4.1	24.38	0.686	0.267	2.1	21.39	0.793	0.243	5.1	21.73	0.669	0.387	3.6	22.75	0.705	0.291

TABLE III
MAP QUALITY ON SEVERAL DYNAMIC SEQUENCES IN THE *BONN* DATASET.

	ball			ball2			ps_tk			ps_tk2			ball_tk			Avg.		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
SplaTAM [6]	17.59	0.766	0.244	16.81	0.650	0.332	18.90	0.655	0.270	17.25	0.721	0.263	15.55	0.633	0.413	17.22	0.685	0.304
MonoGS [9]	17.72	0.712	0.478	19.44	0.747	0.367	18.8	0.736	0.399	20.01	0.755	0.375	18.89	0.623	0.272	18.97	0.715	0.378
Ours	27.89	0.857	0.236	29.65	0.839	0.272	27.66	0.832	0.265	31.18	0.876	0.259	27.19	0.865	0.264	28.71	0.854	0.259

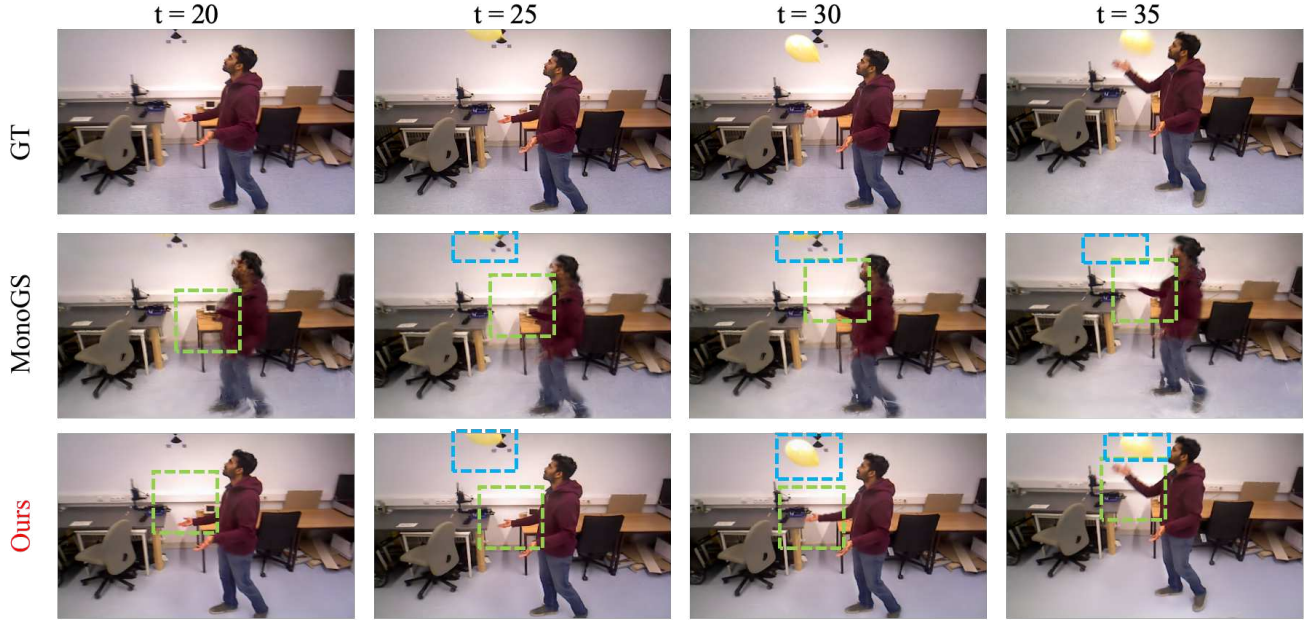


Fig. 6. Visual comparison of the rendering image on the *BONN* datasets.

Structural Similarity Index (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS).

We run our SLAM on a laptop with an Intel Core i7-13700HX CPU and a single NVIDIA GeForce RTX 4080 GPU. The track filtering parameters are set at $\gamma_v = 0.9, \gamma_d = 0.9, \gamma_u = 0.8, \gamma_{track} = 3$. Experimental results show that our algorithm can run at 1.5 FPS on the BONN dataset and 0.9 FPS on the TartanAir-Shibuya dataset, achieving SOTA performance in both tracking and mapping. Our primary comparisons are conducted against 3DGS-based methods like DG-SLAM [22], MonoGS [9], GS-SLAM[24], SplaTAM[6]; additionally, we include comparisons with methods such as Rodyn-SLAM[4], ESLAM[5], Co-SLAM[20], Vox-Fusion[25], NICE-SLAM[30], iMAP*[17], ORB-SLAM3 [2], ReFusion [12] and DROID-VO[19]. It is worth noting that DG-SLAM is a recently introduced method. Since its code has not been open-sourced, the experimental data used in this work are derived from its original paper [22]. More implementation details and evaluations will be published at

this link¹. The specific results are as follows:

B. Evaluation of Camera Tracking Performance

We evaluate our method on both controlled laboratory scenes (BONN dataset) and complex outdoor dynamic environments (TartanAir-Shibuya). As shown in Table I, our approach achieves superior camera pose accuracy compared to state-of-the-art SLAM systems across various dynamic scenarios. Notably, we reduce ATE RMSE by 7.3% compared to the closest competitor DG-SLAM [22] (5.1 cm vs. 5.5 cm), demonstrating the effectiveness of our spatio-temporal Gaussian representation in handling persistent motions. Traditional geometric methods like ORB-SLAM3 [2] exhibit significant failures in high-dynamic sequences (e.g., 77.9 cm ATE RMSE in ps_tk2), while neural implicit approaches (NICE-SLAM, Vox-Fusion) suffer from catastrophic tracking failures due to their static scene assumptions.

The advantages of our method become more pronounced in large-scale dynamic environments, as evidenced by the

¹<https://github.com/zhicongsun/D4DGS-SLAM>

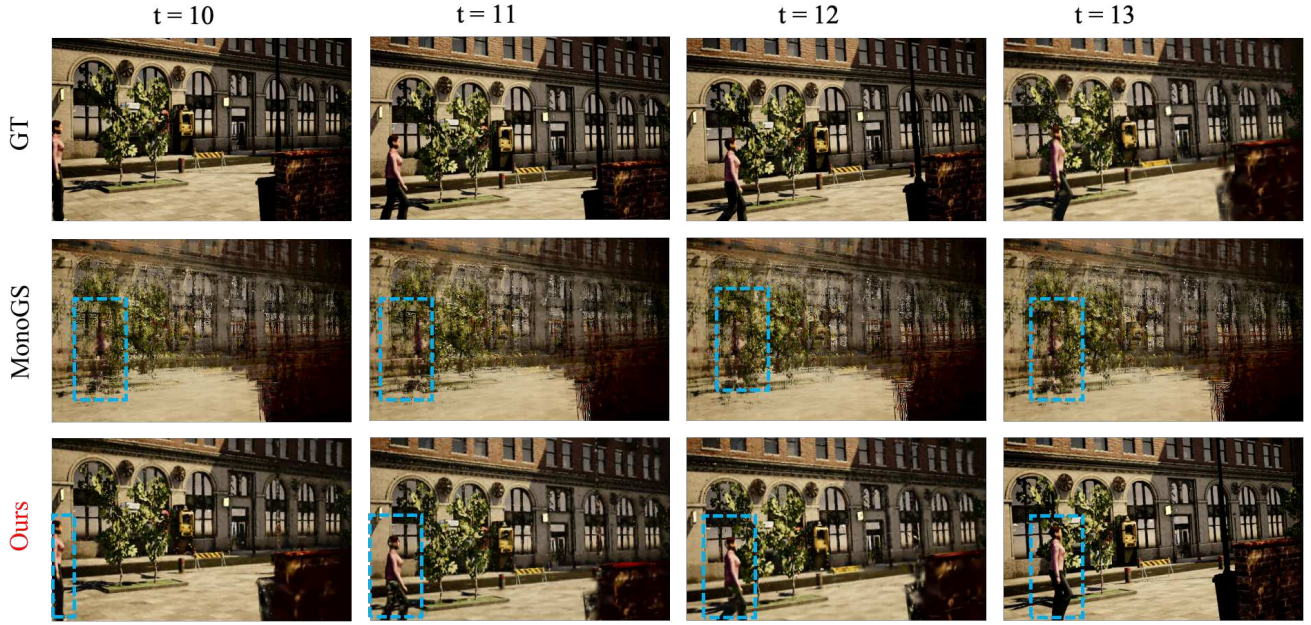


Fig. 7. Visual comparison of the rendering image on the *TartanAir-Shibuya* datasets.

TartanAir-Shibuya results in Table II. Our system achieves a 95.1% reduction in average ATE compared to SplatAM [6] (3.6 cm vs. 73.2 cm) while simultaneously improving rendering quality metrics, 83.1% higher PSNR (22.75 vs. 12.43) and 45.9% better SSIM (0.705 vs. 0.384). This dual improvement stems from our method explicitly models object motions rather than treating them as outliers. Particularly in `rc06` with rapid camera movements, our method maintains sub-centimeter tracking accuracy (2.1 cm ATE) where baseline Gaussian SLAM systems fail catastrophically (larger than 80 cm ATE RMSE).

These results validate our key design choices. Our approach demonstrates particular strength in scenes with both static and moving objects, where traditional dynamic SLAM methods struggle to balance reconstruction quality with tracking robustness.

C. Evaluation of Mapping Performance

In terms of mapping performance comparison, we primarily compare with the baseline methods MonoGS [9] and SplatAM [6], which are representative 3DGS-based SLAM. The reason for this is that existing NeRF-based SLAM methods and 3DGS-based methods for dynamic scenes do not have the capability to reconstruct a real map; they choose to filter out dynamic objects. Additionally, the latest DG-SLAM [24] code has not been open-sourced yet, so we are temporarily unable to test its mapping performance. However, since DG-SLAM [24], like MonoGS [9] and SplatAM [6], essentially uses 3DGS for mapping, comparing our proposed dynamic map representation method with these static 3DGS methods can still reveal the advantages and disadvantages of our approach.

As shown in Table III, our method achieves 28.71 PSNR, 0.854 SSIM, and 0.259 LPIPS on average, outperforming SplatAM and MonoGS by significant margins (e.g. +10.5

PSNR and -0.15 LPIPS). The superiority is consistent across all sequences involving moving objects like ball, demonstrating 4DGS's ability to model spatio-temporal variations without ghosting artifacts. Similar trends hold for the *TartanAir-Shibuya* dataset (Table 2): our approach attains 22.75 PSNR and 0.705 SSIM, surpassing baselines by +8 PSNR and +0.3 SSIM. Notably, the LPIPS score (0.291) is 45% lower than SplatAM, indicating superior perceptual consistency.

We visualize rendered scenes from our method and 3DGS-based MonoGS on BONN and *TartanAir-Shibuya*, respectively. As shown in Fig. 6, our method effectively reconstructs the moving hand (highlighted by the green box) and the balloon (highlighted by the yellow box). In contrast, MonoGS exhibits delayed reconstruction of the moving hand and fails to reconstruct the newly appeared balloon. As shown in the Fig. 7, MonoGS exhibit severe ghosting artifacts, such as semi-transparent duplicates of moving objects, which significantly degrade the quality of the reconstructed scene. In contrast, our method effectively captures dynamic features (e.g., the trajectory of a walking person) while preserving static structures with high fidelity. Furthermore, our results demonstrate smooth transitions between frames (left-to-right in the figures), ensuring temporal consistency in the reconstructed scene. On the other hand, MonoGS suffers from flickering artifacts, which are primarily caused by misestimated Gaussian positions due to dynamic occlusions. This highlights the robustness of our approach in handling dynamic environments compared to traditional 3DGS-based techniques.

D. Ablation study

The ablation study on the ball scene in the *BONN* dataset demonstrates the effectiveness of our proposed method. Without dynamics-aware InfoModule, the system exhibits significantly higher ATE (27.9 cm) and lower PSNR (20.23),

SSIM (0.790), and LPIPS (0.371), highlighting the critical role of dynamics-aware InfoModule in improving tracking accuracy and reconstruction quality. Without 4DGS, the ATE improves to 7.2 cm, but PSNR (18.23), SSIM (0.645), and LPIPS (0.327) remain suboptimal, indicating 4DGS's importance for high-fidelity reconstruction. Our full method achieves the best performance, with an ATE of 3.6 cm, PSNR of 27.89, SSIM of 0.857, and LPIPS of 0.236, confirming the synergistic benefits of integrating LEAP and 4DGS for dynamic scene understanding.

TABLE IV

ABLATION STUDY ON THE BALL SCENE IN THE BONN DATASET.

D-AWARE MEANS THE DYNAMICS-AWARE INFOMODULE

	ATE RMSE↓	PSNR↑	SSIM↑	LPIPS↓
w/o D-aware	27.9	20.23	0.790	0.371
w/o 4DGS	7.2	18.23	0.645	0.327
Ours	3.6	27.89	0.857	0.236

V. CONCLUSIONS

In this paper, we present D4DGS-SLAM, the first SLAM method based on dynamic map representation and dynamics-aware module. Experimental results on dynamic datasets demonstrate that our method effectively captures the dynamic characteristics of the scene, outperforming state-of-the-art algorithms in both camera tracking accuracy and mapping quality. In addition to introducing 4DGS into map representation for the first time, we also provide a framework that integrates dynamic information into 4D Gaussian optimization. This will allow our algorithm to improve as dynamic point tracking methods advance. Nonetheless, this work serves as a preliminary investigation, with many aspects warranting further study and the potential to reveal additional challenges. For example, tracking performance may degrade under heavy occlusion by dynamic objects, due to inherent difficulties in distinguishing between dynamic and static scene components. Moreover, the current reliance on a large number of Gaussians to represent scenes restricts the applicability of our algorithm on resource-constrained platforms, such as small robots or drones. Future work will aim to explore these and other limitations in greater depth, and to develop effective solutions accordingly.

REFERENCES

- [1] B. Bescos, J. M. F  cil, J. Civera, and J. Neira, "Dynaslam: Tracking, mapping, and inpainting in dynamic scenes," in *IEEE Robotics and Automation Letters*, vol. 3, no. 4. IEEE, 2018, pp. 4076–4083.
- [2] C. Campos, R. Elvira, J. J. G. Rodr  guez, J. M. Montiel, and J. D. Tard  s, "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam," 2021.
- [3] W. Chen, L. Chen, R. Wang, and M. Pollefeys, "Leap-vo: Long-term effective any point tracking for visual odometry," 2024.
- [4] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, "Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields," *IEEE Robotics and Automation Letters*, 2024.
- [5] M. M. Johari, C. Carta, and F. Fleuret, "Eslam: Efficient dense slam system based on hybrid representation of signed distance fields," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 17 408–17 419.

- [6] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "Splatam: Splat, track & map 3d gaussians for dense rgb-d slam," in *CVPR*, 2024.
- [7] B. Kerbl, G. Kopanas, T. Leimk  hler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [8] M. Li, J. He, G. Jiang, and H. Wang, "Ddn-slam: Real-time dense dynamic neural implicit slam with joint semantic encoding," *arXiv preprint arXiv:2402.03246*, 2024.
- [9] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian splatting slam," in *CVPR*, 2024.
- [10] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, 2021.
- [11] R. Mur-Artal and J. D. Tard  s, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," 2017.
- [12] E. Palazzolo, J. Behley, P. Lottes, P. Gigu  re, and C. Stachniss, "ReFusion: 3D Reconstruction in Dynamic Environments for RGB-D Cameras Exploiting Residuals," *arXiv*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.02082>
- [13] Y. Qiu, C. Wang, W. Wang, M. Henein, and S. Scherer, "Airdos: Dynamic slam benefits from articulated objects," in *ICRA*. IEEE, 2022, pp. 8047–8053.
- [14] C. Ruan, Q. Zang, K. Zhang, and K. Huang, "Dn-slam: A visual slam with orb features and nerf mapping in dynamic environments," *IEEE Sensors Journal*, vol. 24, no. 4, pp. 5279–5287, 2023.
- [15] E. Sandstr  m, Y. Li, L. Van Gool, and M. R. Oswald, "Point-slam: Dense neural point cloud-based slam," in *Int. Conf. Comput. Vis.*, 2023.
- [16] E. Sucar, S. Liu, J. Ortiz, and A. Davison, "iMAP: Implicit mapping and positioning in real-time," in *Int. Conf. Comput. Vis.*, 2021.
- [17] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "imap: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 6229–6238.
- [18] Z. Teed and J. Deng, "Raft: Recurrent all-pairs field transforms for optical flow," in *ECCV*. Springer, 2020, pp. 402–419.
- [19] —, "Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras," in *Neural Information Processing Systems*, 2021.
- [20] H. Wang, J. Wang, and L. Agapito, "Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 13 293–13 302.
- [21] H. Xu, J. Zhang, J. Cai, H. Rezaatofighi, and D. Tao, "Gmflow: Learning optical flow via global matching," in *Proceedings of the CVPR*, 2022, pp. 8121–8130.
- [22] Y. Xu, H. Jiang, Z. Xiao, J. Feng, and L. Zhang, "DG-SLAM: Robust dynamic gaussian splatting SLAM with hybrid pose optimization," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [23] Z. Xu, J. Niu, Q. Li, T. Ren, and C. Chen, "Nid-slam: Neural implicit representation-based rgb-d slam in dynamic environments," in *2024 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2024, pp. 1–6.
- [24] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "Gs-slam: Dense visual slam with 3d gaussian splatting," in *CVPR*, 2024.
- [25] X. Yang, H. Li, H. Zhai, Y. Ming, Y. Liu, and G. Zhang, "Vox-fusion: Dense tracking and mapping with voxel-based neural implicit representation," in *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2022, pp. 499–507.
- [26] Z. Yang, H. Yang, Z. Pan, and L. Zhang, "Real-time photorealistic dynamic scene representation and rendering with 4d gaussian splatting," in *International Conference on Learning Representations (ICLR)*, 2024.
- [27] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, "Ds-slam: A semantic visual slam towards dynamic environments," in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018.
- [28] J. Zhang, M. Henein, R. Mahony, and V. Ila, "Vdo-slam: a visual dynamic object-aware slam system," *arXiv preprint*, 2020.
- [29] T. Zhang, H. Zhang, Y. Li, Y. Nakamura, and L. Zhang, "Flowfusion: Dynamic dense rgb-d slam based on optical flow," in *IEEE Int. Conf. Robot. Autom.*, 2020.
- [30] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "Nice-slam: Neural implicit scalable encoding for slam," in *IEEE Conf. Comput. Vis. Pattern Recog.*, 2022.